Addressing Imbalance in Multitask Learning

Logan Rose and Gabrelle Duenas

[1] The University of Ottawa
lrose039@uottawa.ca
[2] gduen055@uottawa.ca

Abstract. The problem of class imbalance presents a significant challenge to the formulation of machine learning models. This challenge is further complicated when formulating multi-task machine learning models. This is because the oversampling of one target class may result in a further degradation performance for the classification of other target classes. In this paper, a strategy is proposed to deal with the class imbalance in multi-task learning. The strategy entails the successive application of SMOTE to resolve the imbalance of each target variable. When oversampling for a given target variable, the other target variables' values are treated as features.

Keywords: Multitask Learning · Class Imbalance · SMOTE · Dataset Complexity.

1 Implemented Solutions

1.1 Problem Description

Most Machine learning models are trained using a large dataset with each entry labeled with its corresponding class. The model then uses this training data to recognize patterns and characteristics native to each class, allowing it to predict the classes of new, unlabeled entries. When training a machine learning model, one major challenge encountered is class imbalance. Class imbalance occurs when the data points of the majority class vastly outnumber the data points of the minority class or classes, as seen in Fig. 1. Due to this imbalance, a machine learning model will be far better at classifying samples from the oversampled class, and will perform comparatively poorly when classifying the minority class. This bias severely hinders the performance of the model.

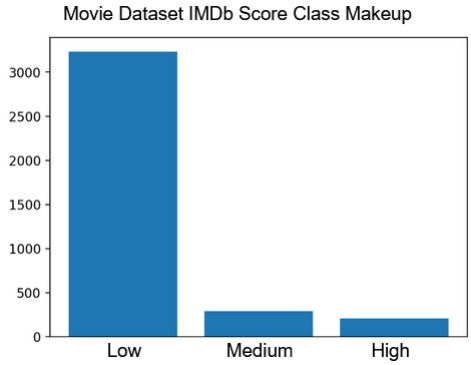

Fig. 1: Class imbalanced visualized. The IMDb Score class has far more entries classified as 'Low' compared to 'Medium' or 'High.'

The most intuitive solution to the class imbalance problem is the resampling of training data, however, this is not always feasible. Solutions to class imbalance that do not involve resampling generally fall into one of two categories: undersampling and oversampling, both of which are visualized in Fig. 2. Undersampling techniques involve ignoring instances of the majority class so the mode has a smaller total number of samples, but the sample data it does have is more balanced. Oversampling involves the creation of synthetic data in the minority class.

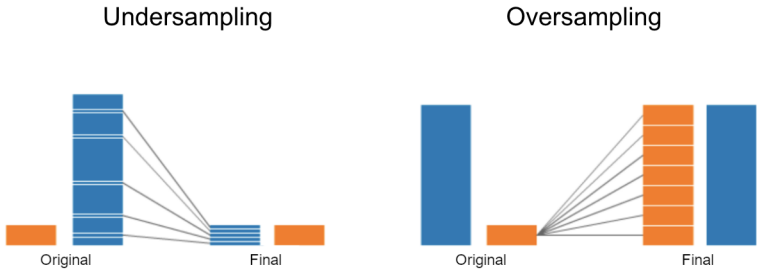

Fig. 2: Oversampling and Undersampling techniques visualized. [10]

The challenges presented by imbalanced data are compounded when training a multitask machine learning model. Multi-Task models seek to classify multiple target variables. Applying oversampling or undersampling techniques to resolve the imbalance present in one target variable may make the imbalance of another target variable worse, exemplified in Fig. 3 where movies classified as minority classes in IMDb Score could also be classified as the majority class in Box Office Gross. The objective of this project is to generate new data for training multitask models effectively for all variables.

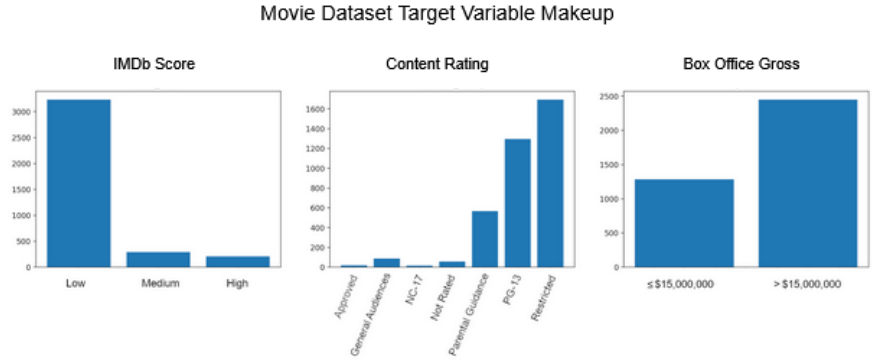

Fig. 3: The intricacies of class imbalance grow as the number of variables grows. Not all movies classified as 'Low' make more than $15 000 000.

1.2 Proposed Procedure

To observe the effects of resampling on a multitask machine learning model, a learner was trained to classify 3 target variables in each dataset. To address the class imbalance present in each of the targets, An oversampling approach known as Synthetic Minority Oversampling Technique (SMOTE) was applied. This technique generates synthetic data for the minority class [6], and was chosen for its higher performance compared to traditional oversampling. [1] This process is done repeatedly, each time addressing the imbalance of a different target variable. On the first pass, Target One is used as the imbalance class, while the values for Target Two and Target Three are treated as features by the SMOTE algorithm. On the second pass, the values for Target One and Target Three, including the newly generated entries from the first pass, are treated as features, while Target Two is passed to SMOTE as the imbalance class. On the third pass, Target One and Target Two are treated as features while Target Three is treated as the imbalance class.

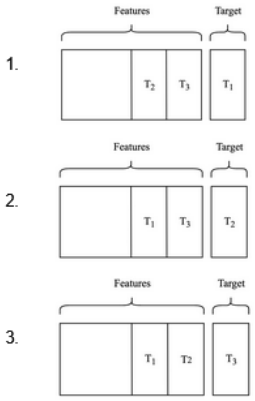

Fig. 4: SMOTE process visualized. Each subsequent iteration builds off of the previous, generating large amounts of synthetic data.

The performance of each learner was quantified using 5-fold cross-validation. In 5-fold cross-validation, the dataset is divided into five equally sized subsets. Then, four subsets (or folds) are used to train a model, while the final subset is used to test the performance of the model. This process is repeated four more times, each time using a different fold to test and the remaining four to train. The average value of each metric across all five folds is then calculated as the performance of the whole dataset. By performing cross validation, a more accurate understanding of the models performance can be calculated, as it overcomes the risk of selecting a lucky or unlucky subset to test the model on.

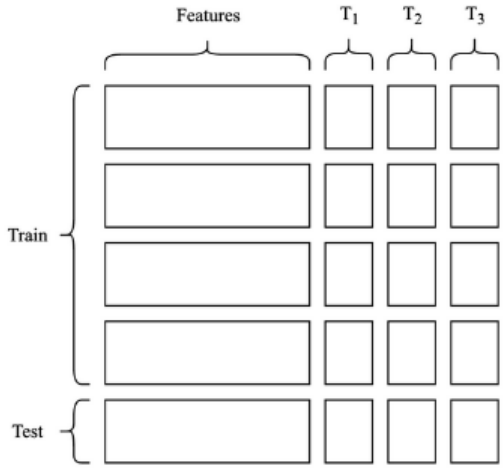

Fig. 5: Cross validation visualized. Each iteration, a new subsection is used as testing data.

2 Methodology

2.1 Datasets

Ten datasets were used during the procedure. There is minimal overlap in dataset subject matter, increasing the diversity of the training data. A table containing each dataset and the three targets classified can be found in Figure 6.

2.2 Experimental Settings

Machine Learning Techniques Used For this procedure, the performance of each of the monitored metrics was reported as the average value across the following 5 machine learning algorithms: Gaussian Naive Bayes, Linear Discriminant Analysis, K Neighbors Classifier, Decision Tree Classifier, and Logistic

Dataset	Target 1	Target 2	Target 3
Banking Data	y	loan	housing
Census Data	Income	Maritial Status	Work Class
Flight Data	Satisfaction	Customer Type	Class
Anuran	Family	Genus	Species
IMDb Movies	IMDb Score	Content Rating	Gross
Mushrooms	Class	Population	Habitat
Online Shopping Intention	Revenue	Visitor Type	Special Day
Paris Housing Data	Category	Is Newly Built	Has Storage Room
Smoking Health Effects	If a person is a smoker	Tartar level	Dental caries
Telco	Churn	Contract	Payment Method

Table 1: Each Dataset and its target variables.

Regression. The Scikit-Learn implementation with default parameters of each algorithm was used in conjunction with the Scikit-Learn MultiOutputClassifier to train each model to classify multiple outputs. [2] The Imbalance-Learn implementation of SMOTE was used with default parameters to oversample each target variable. [7]

Measures Tracked

Precision, Recall, F1 Score: Precision, recall, and specificity are commonly used measures of performance in machine learning and data analysis. These metrics evaluate the accuracy of a model in making predictions or classifications. Precision measures the proportion of positive predictions that are actually correct [3], while recall measures the proportion of actual positive cases that are correctly predicted by the model [4]. F1 score is calculated as the harmonic mean of precision and recall, with a higher score indicating better performance [5]. The values for each of these metrics were calculated using the Scikit-Learn module's implementation. [2]

Specificity, Geometric Mean: The geometric mean is an informative metric used to evaluate the performance of multi-class classification models. It is calculated as the n^{th} root of the product of class-wise sensitivity for n classes. [8] The geometric mean is particularly useful because it gives equal weight to each class, making it a suitable metric for evaluating models that need to classify multiple variables. Specificity measures the proportion of negative cases that are correctly predicted [9]. Geometric Mean and Specificity were both calculated using tracked using the Imblearn module's implementation.

Smote Parameters

For the implementation of SMOTE, a singular seed was created and used whenever a model was trained. This ensures that the randomness that one learner

experiences is shared throughout all learners. While each execution remains random, all learners within that execution are random in the same way, removing any influence that randomness may grant. The parameter chosen for sampling strategy was 'not majority', indicating that SMOTE would be applied to every class except for the majority class. This parameter was chosen over 'minority' due to the multiclass nature of the datasets used. [7]

Seed and Environment

A seed of 42 was used for all models. The code was executed in Jupyter notebooks and can be found at `https://github.com/Logan-Rose/CSI4900`. Matplotlib was used to generate the figures presented in this paper. As previously mentioned, Scikit-Learn and Imbalance-Learn utility functions were used heavily in the conducting of these experiments.

3 Results

3.1 Performance

The first, and perhaps the most important observation, is that repeatedly applying SMOTE does not appear to significantly degrade performance in any of the metrics observed. The impact on Specificity was negligible, averaging a performance increase of 0.6% with no observed correlation between the sort order of the targets and the performance difference. The impact on F1 was negligible, with performance the average performance increasing by 0.4 or decreasing by 0.5% with no observed correlation between the sort order of the targets and the performance difference. A slight performance improvement was observed in model recall, increasing by an average of just over 2% with no observed correlation between the sort order of the targets and the performance difference. A slight performance degradation was observed in model precision, decreasing by an average of between 0.6% and 1.7%. A significant performance improvement was observed in the measurement of the geometric mean. On average, the geometric mean score of the model improved by 7.9%.

3.2 Dataset Size

As SMOTE is an oversampling technique, its application results in increased dataset size. While a single smote application results in an average increase of 1.5x in the number of sample sizes for the datasets used in the experiment, applying smote three times results in an 8x increase in sample size. (Appendix 4.6)

3.3 Impact of target complexity on classification

The complexity of a given target variable has a significant impact on the initial performance of a machine learning model before taking imbalance, and subsequent oversampling into account. The table below depicts the baseline performance metrics (before any oversampling is applied) for the most complex target in each dataset, compared to the baseline performance metrics for the least complex target in each dataset.

Average Initial Performance for Most and Least Complex Target Variables

Metric	High Complexity	Low Complexity
Precision	49.8	71.6
Recall	50.1	69.5
Specificity	74.1	73.0
F1	47.5	68.6
Geometric Mean	35.8	57.1

3.4 Performance on high and low complexity targets

In addition to having a higher baseline performance, low-complexity target variables responded better to oversampling when compared to high-complexity targets. After applying smote to address the imbalance in each of the target variables. It was observed that in the Anuran dataset, which has particularly low complexity for all three target variables, the models were able to improve significantly upon an already well-performing baseline. While there was some variance in performance (¡ 2%) depending on sort order, the model's ability to classify the Family variable saw a 24% improvement, Genus improved by 18%, and Species classification improved by 6.4%. While these results are impressive, more datasets with similar complexity would need to be observed to verify these observations.

3.5 The effect of applying oversampling to targets in order of target complexity

A marginal performance improvement in Geometric Mean was observed when the target variables are oversampled in order from least complex to most complex, while a marginal performance decrease was observed when the target variables are oversampled in the reverse order. When targets are oversampled in a random order, the average increase in geometric mean was 7.9%. When targets are oversampled in order of increasing complexity, the average performance increase in Geometric mean was 8.5%. When targets are oversampled in order of decreasing complexity, the average performance increase in Geometric mean was 7.4%.

References

[1] Rok Blagus and Lara Lusa. "SMOTE for high-dimensional class-imbalanced data". In: *BMC bioinformatics* 14.1 (2013), pp. 1–16.

[2] Lars Buitinck et al. "API design for machine learning software: experiences from the scikit-learn project". In: *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*. 2013, pp. 108–122.

[3] Lars Buitinck et al. "API design for machine learning software: experiences from the scikit-learn project". In: *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*. 2013, pp. 108–122. URL: `https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html`.

[4] Lars Buitinck et al. "API design for machine learning software: experiences from the scikit-learn project". In: *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*. 2013, pp. 108–122. URL: `https://scikit-learn.org/stable/modules/generated/sklearn.metrics.recall_score.html`.

[5] Lars Buitinck et al. "API design for machine learning software: experiences from the scikit-learn project". In: *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*. 2013, pp. 108–122. URL: `https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html`.

[6] Nitesh V Chawla et al. "SMOTE: synthetic minority over-sampling technique". In: *Journal of artificial intelligence research* 16 (2002), pp. 321–357.

[7] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning". In: *Journal of Machine Learning Research* 18.17 (2017), pp. 1–5. URL: `http://jmlr.org/papers/v18/16-365.html`.

[8] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning". In: *Journal of Machine Learning Research* 18.17 (2017), pp. 1–5. URL: `https://imbalanced-learn.org/stable/references/generated/imblearn.metrics.geometric_mean_score.html`.

[9] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning". In: *Journal of Machine Learning Research* 18.17 (2017), pp. 1–5. URL: `https://imbalanced-learn.org/stable/references/generated/imblearn.metrics.specificity_score.html`.

[10] Nour Al-Rahman Al-Serw. "Undersampling and oversampling: An old and a new approach". In: (). URL: `https://medium.com/analytics-vidhya/undersampling-and-oversampling-an-old-and-a-new-approach-4f984a0e8392`.

4 Appendices

4.1 Appendix I

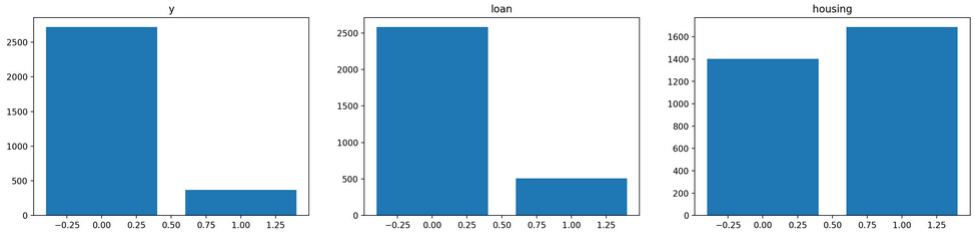

Fig. 6: Class distribution of the 3 variables used in the procedure belonging to the Banking Data dataset.

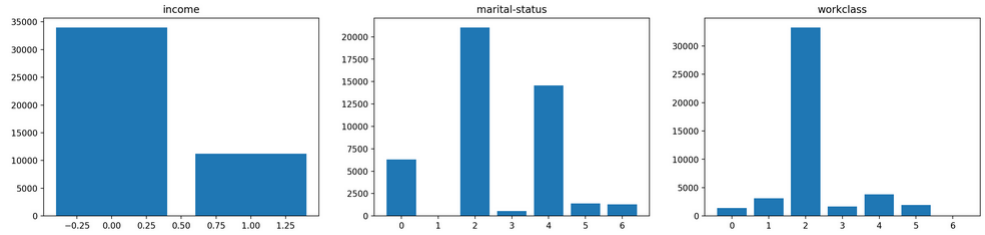

Fig. 7: Class distribution of the 3 variables used in the procedure belonging to the Census Data dataset.

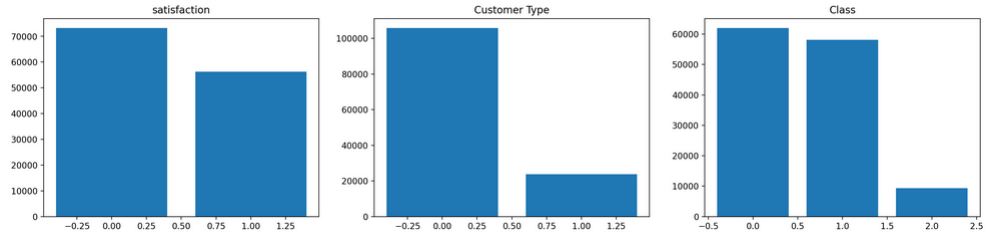

Fig. 8: Class distribution of the 3 variables used in the procedure belonging to the Flight Data dataset.

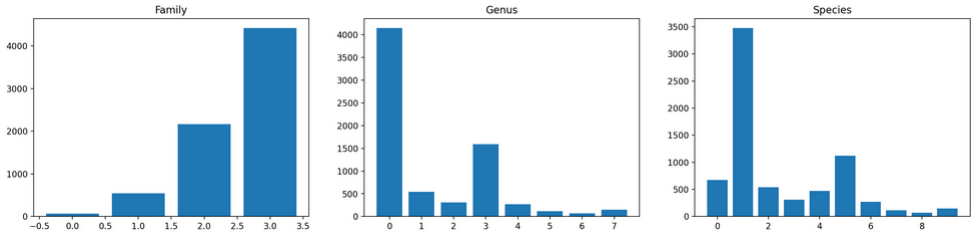

Fig. 9: Class distribution of the 3 variables used in the procedure belonging to the Anuran dataset.

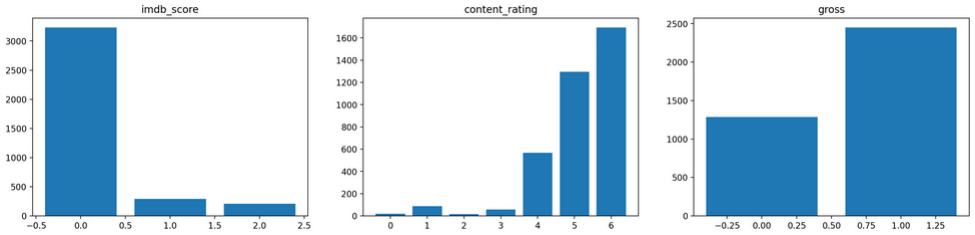

Fig. 10: Class distribution of the 3 variables used in the procedure belonging to the IMDb movies dataset.

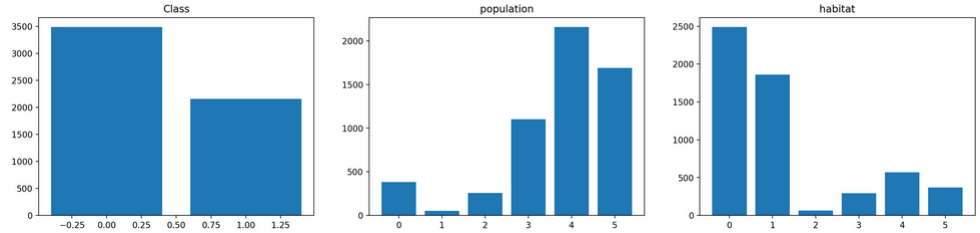

Fig. 11: Class distribution of the 3 variables used in the procedure belonging to the Mushroom dataset.

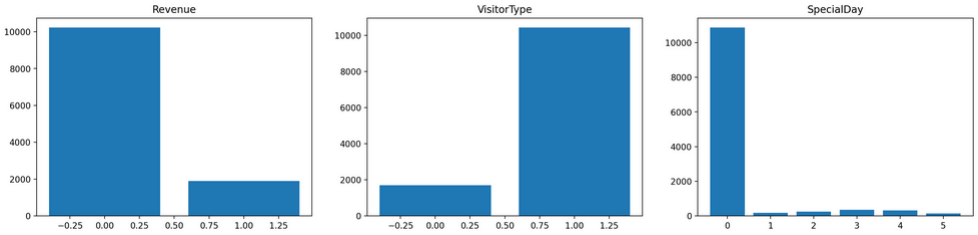

Fig. 12: Class distribution of the 3 variables used in the procedure belonging to the Online Shopping Intention dataset.

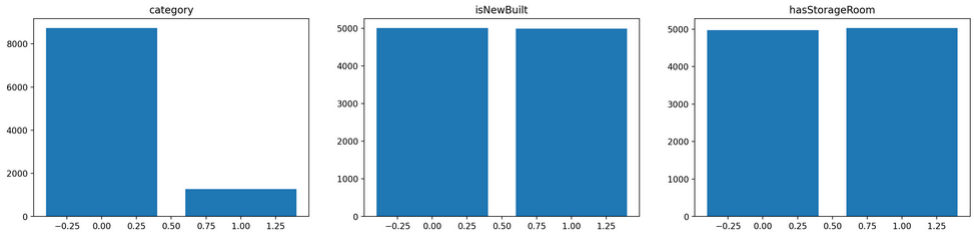

Fig. 13: Class distribution of the 3 variables used in the procedure belonging to the Paris Housing Data dataset.

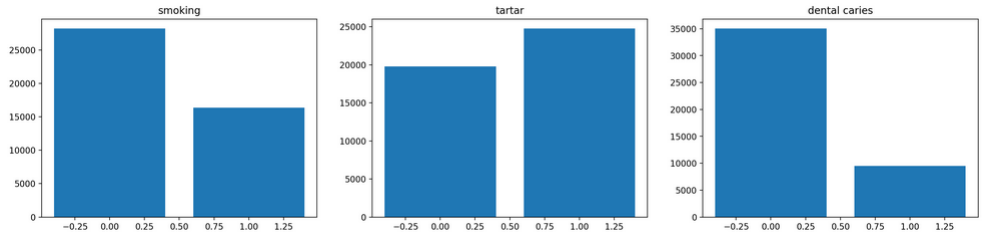

Fig. 14: Class distribution of the 3 variables used in the procedure belonging to the Smoking Health Effects dataset.

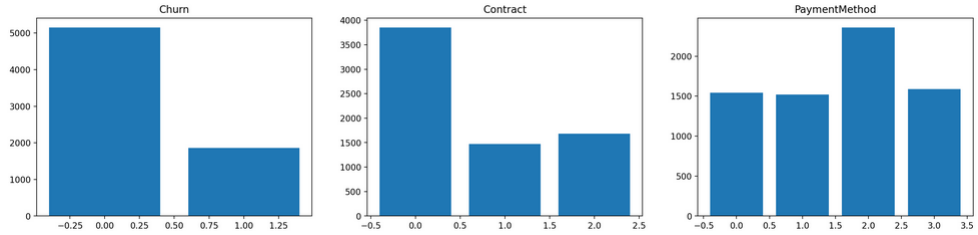

Fig. 15: Class distribution of the 3 variables used in the procedure belonging to the Telco Data dataset.

4.2 Appendix II

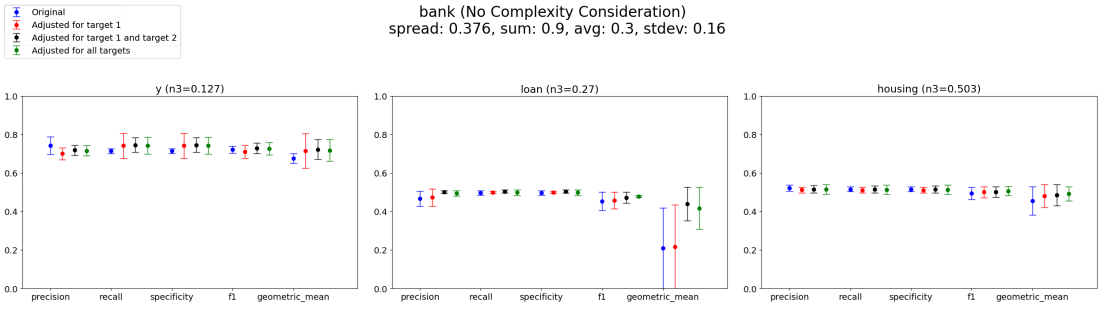

Fig. 16: Performance of the Banking Data model with no consideration for complexity.

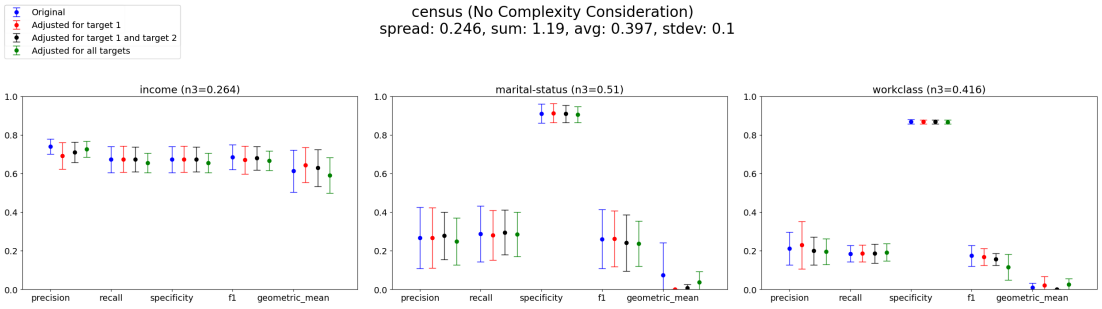

Fig. 17: Performance of the Census Data model with no consideration for complexity.

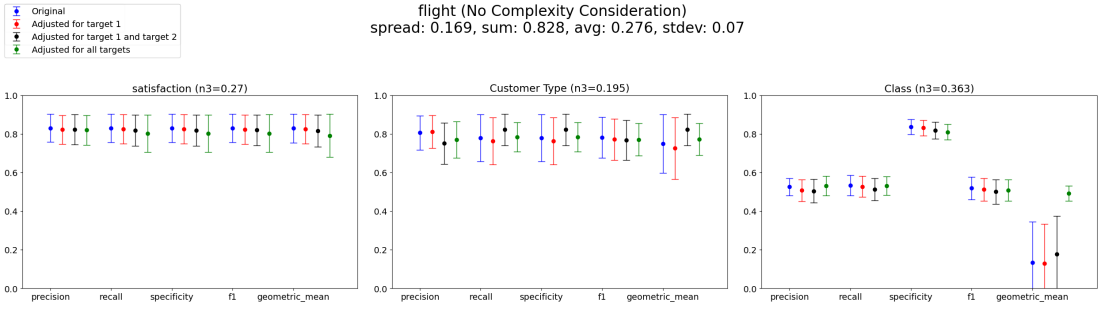

Fig. 18: Performance of the Flight Data model with no consideration for complexity.

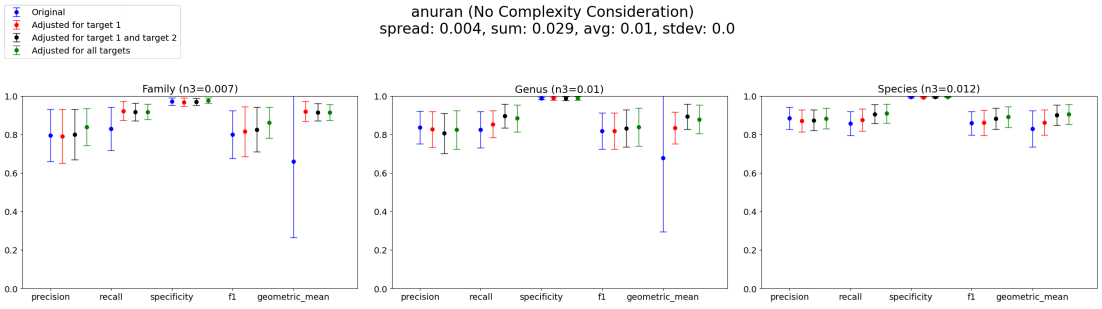

Fig. 19: Performance of the Anuran model with no consideration for complexity.

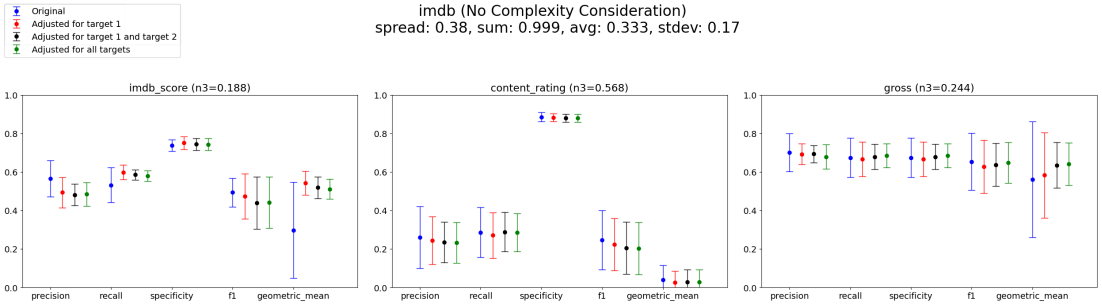

Fig. 20: Performance of the IMDb movies model with no consideration for complexity.

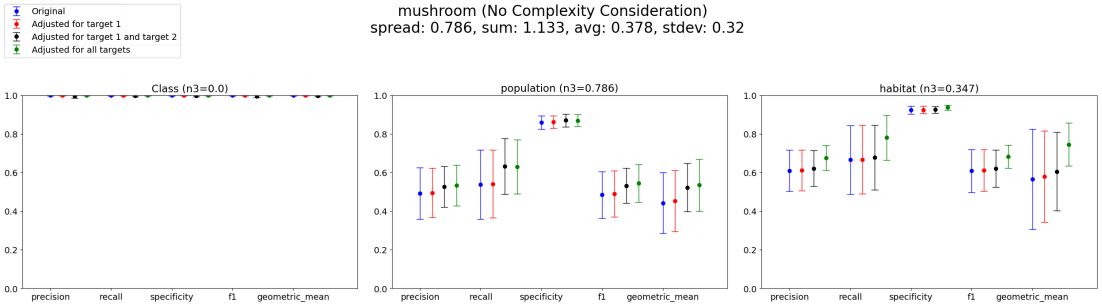

Fig. 21: Performance of the Mushroom model with no consideration for complexity.

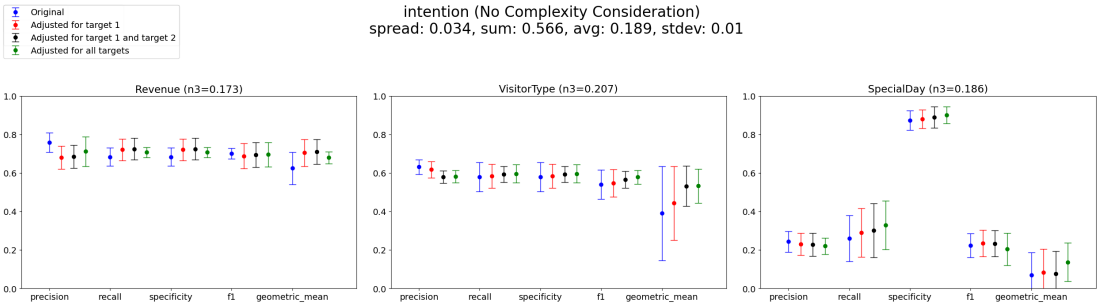

Fig. 22: Performance of the Online Shopping Intention model with no consideration for complexity.

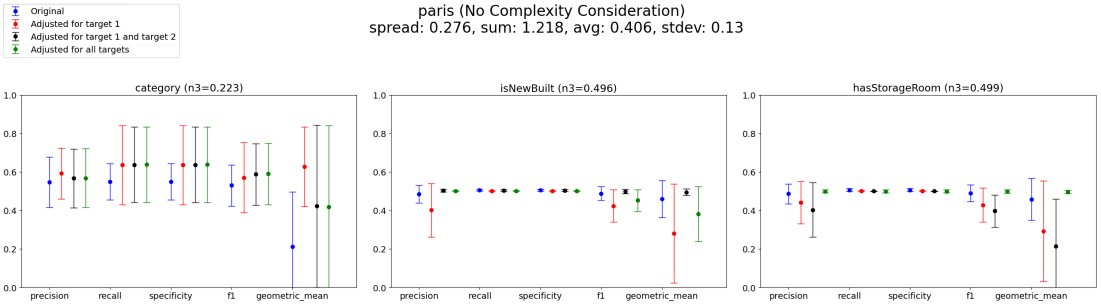

Fig. 23: Performance of the Paris Housing Data model with no consideration for complexity.

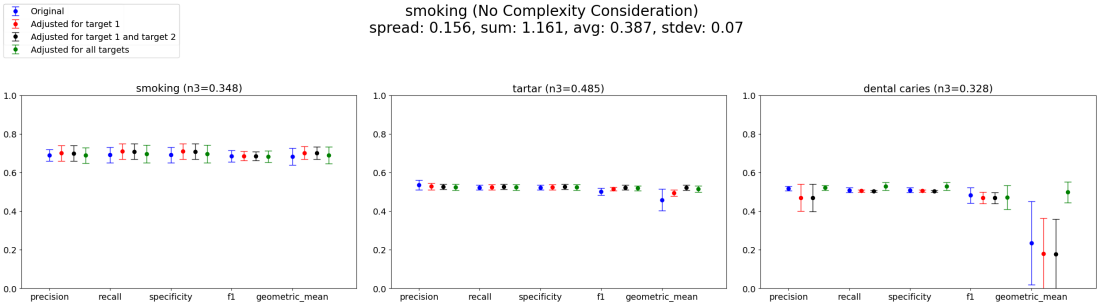

Fig. 24: Performance of the Smoking Health Effects model with no consideration for complexity.

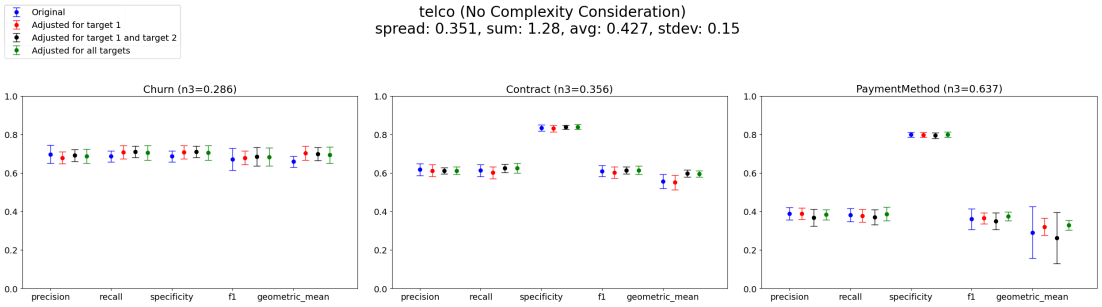

Fig. 25: Performance of the Telco Data model with no consideration for complexity.

4.3 Appendix III

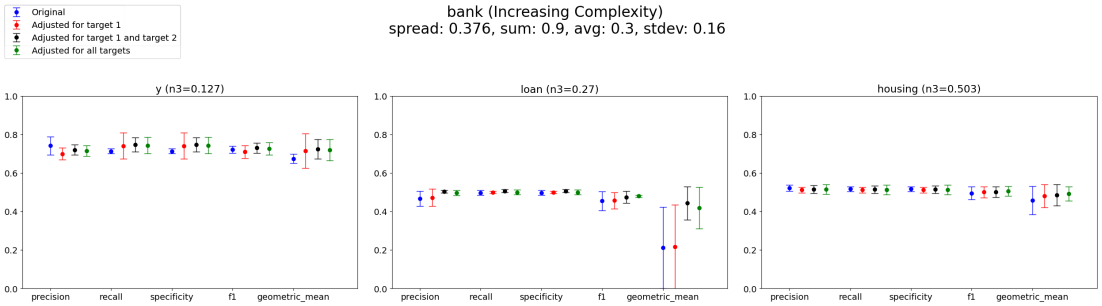

Fig. 26: Performance of the Banking Data model when applying SMOTE in order of ascending complexity.

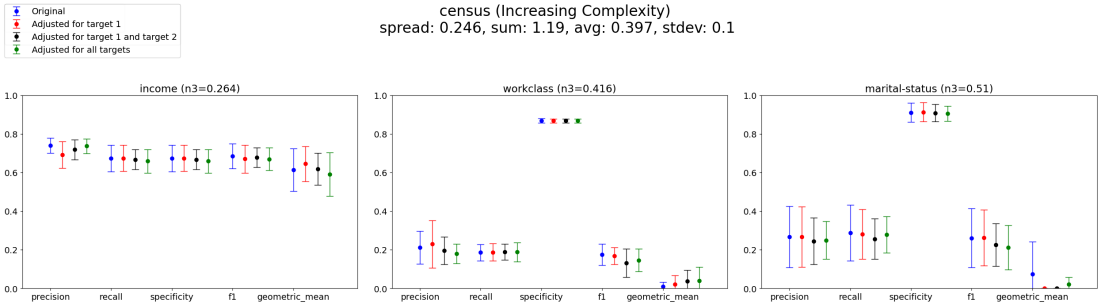

Fig. 27: Performance of the Census Data model when applying SMOTE in order of ascending complexity.

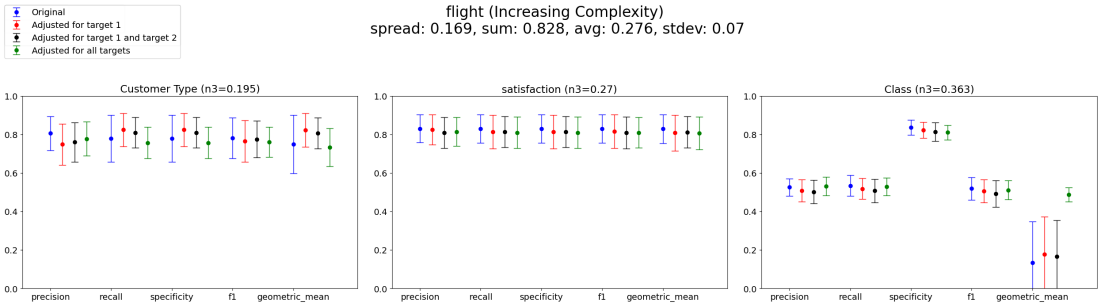

Fig. 28: Performance of the Flight Data model when applying SMOTE in order of ascending complexity.

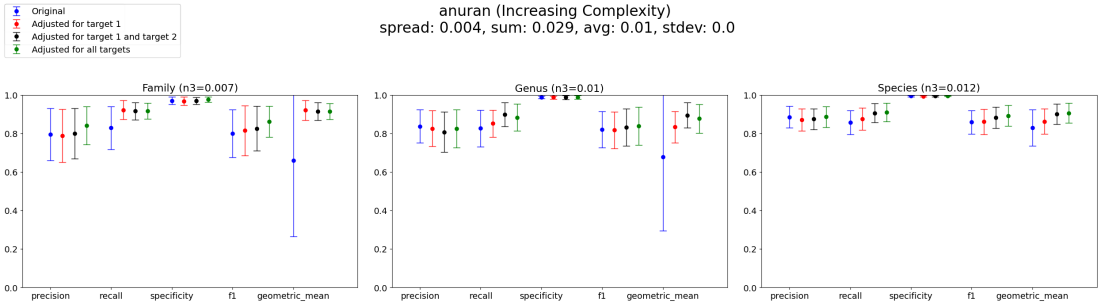

Fig. 29: Performance of the Anuran model when applying SMOTE in order of ascending complexity.

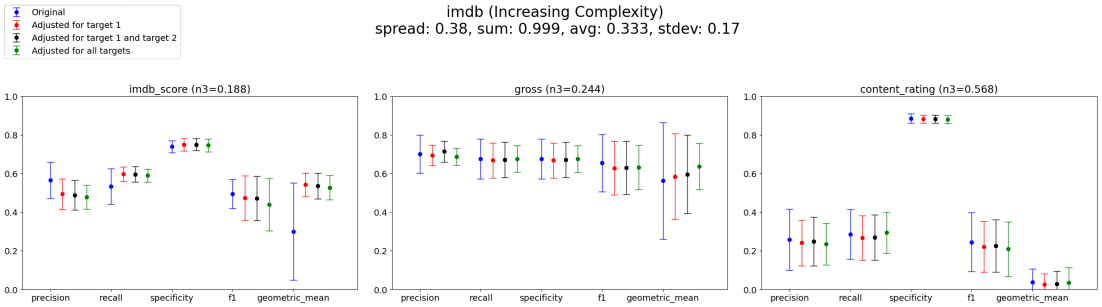

Fig. 30: Performance of the IMDb movies model when applying SMOTE in order of ascending complexity.

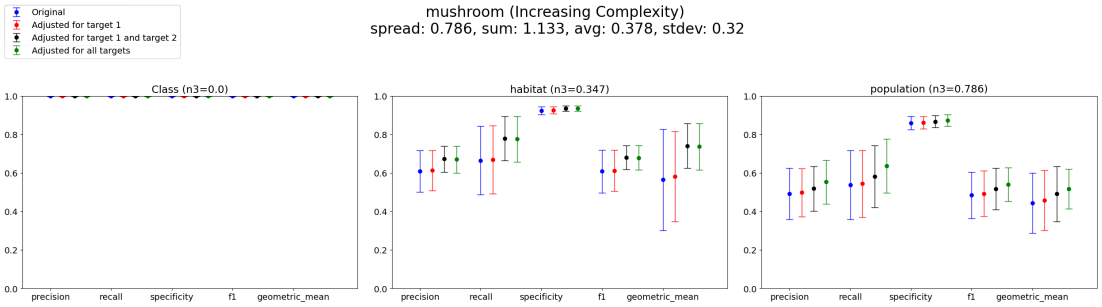

Fig. 31: Performance of the Mushroom model when applying SMOTE in order of ascending complexity.

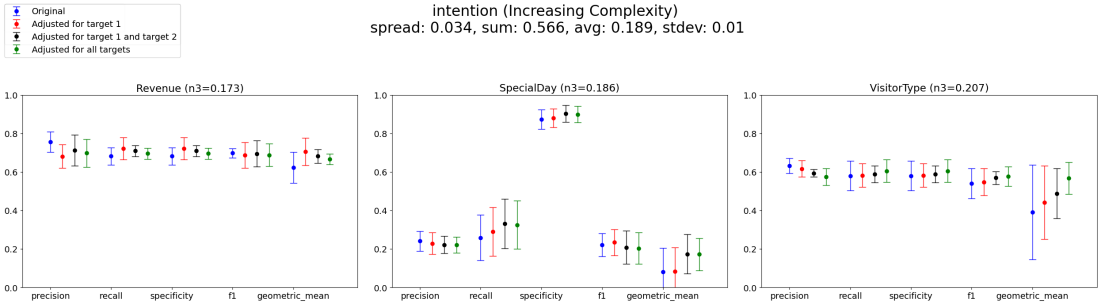

Fig. 32: Performance of the Online Shopping Intention model when applying SMOTE in order of ascending complexity.

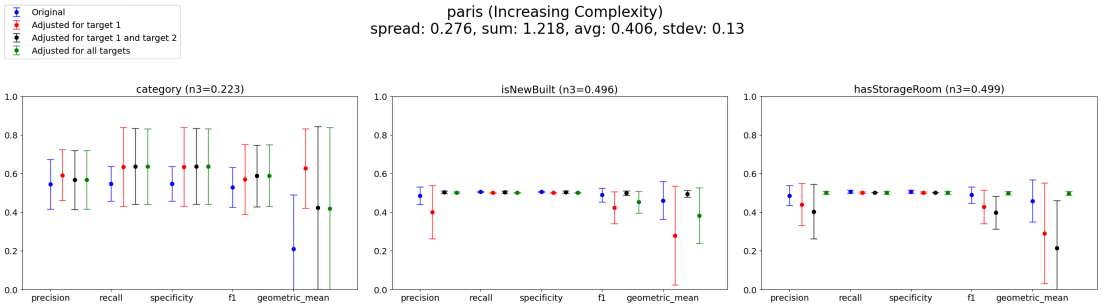

Fig. 33: Performance of the Paris Housing Data model when applying SMOTE in order of ascending complexity.

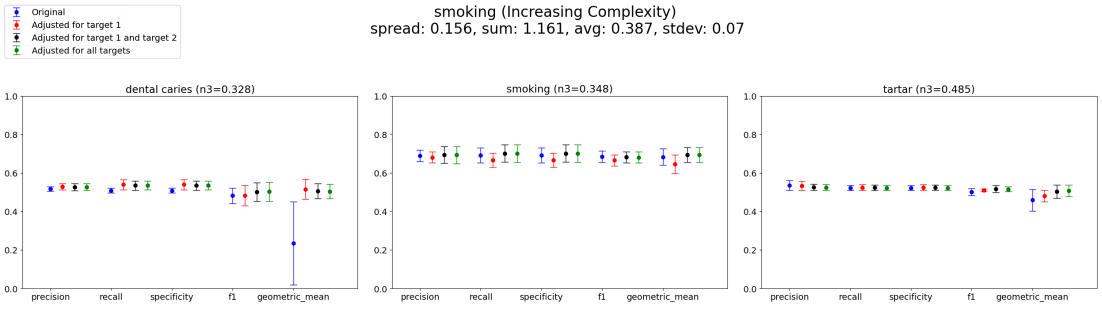

Fig. 34: Performance of the Smoking Health Effects model when applying SMOTE in order of ascending complexity.

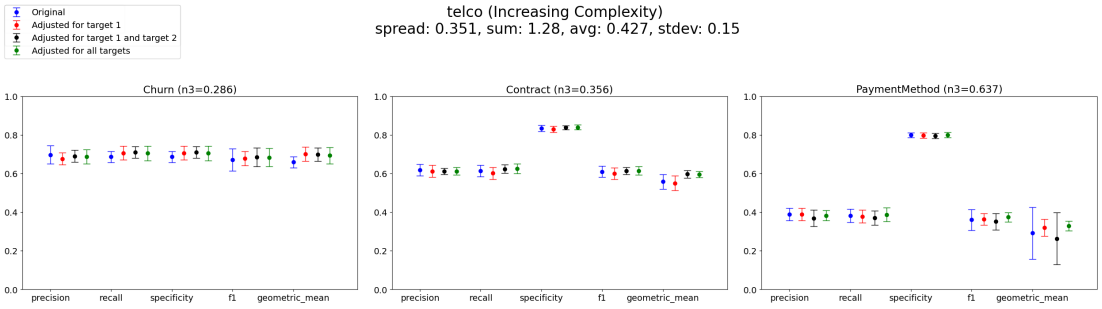

Fig. 35: Performance of the Telco Data model when applying SMOTE in order of ascending complexity.

4.4 Appendix IV

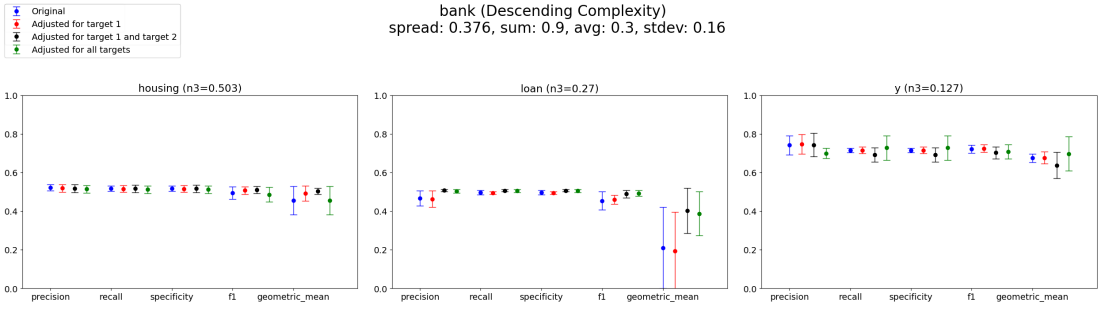

Fig. 36: Performance of the Banking Data model when applying SMOTE in order of descending complexity.

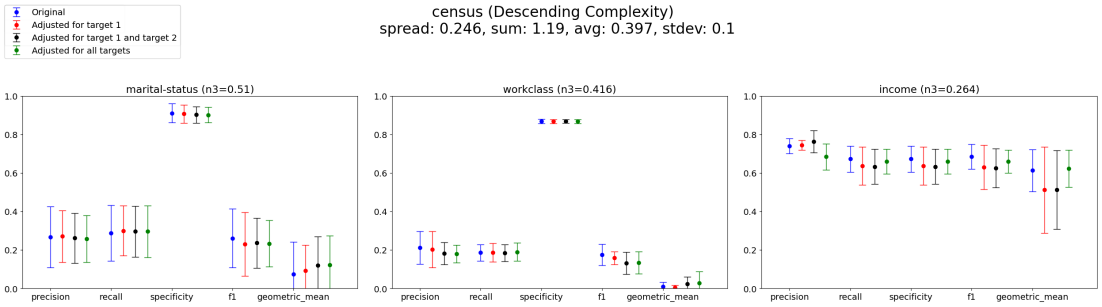

Fig. 37: Performance of the Census Data model when applying SMOTE in order of descending complexity.

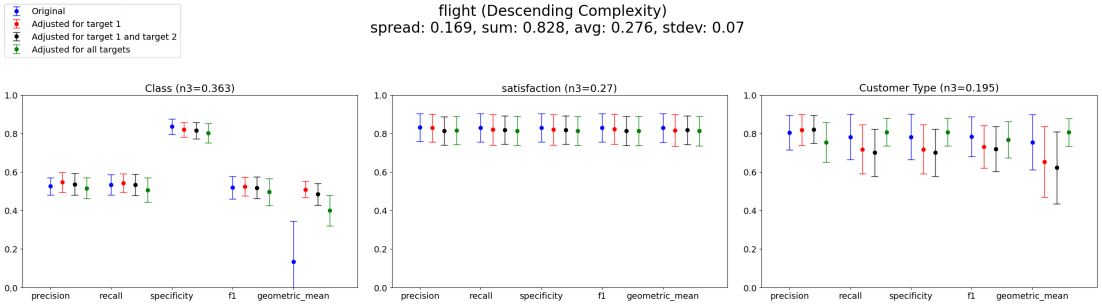

Fig. 38: Performance of the Flight Data model when applying SMOTE in order of descending complexity.

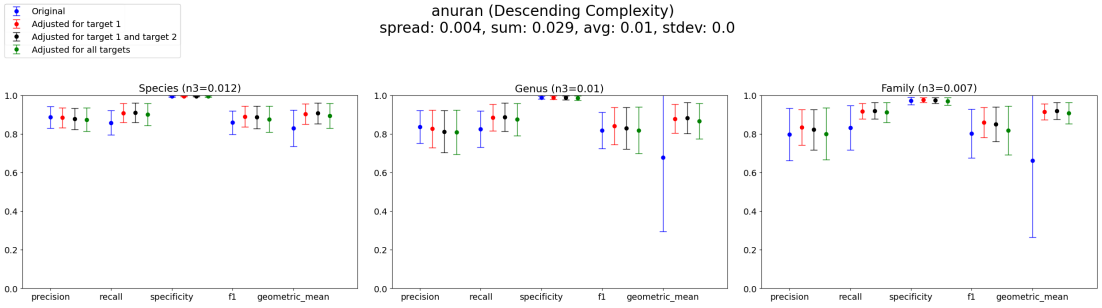

Fig. 39: Performance of the Anuran model when applying SMOTE in order of descending complexity.

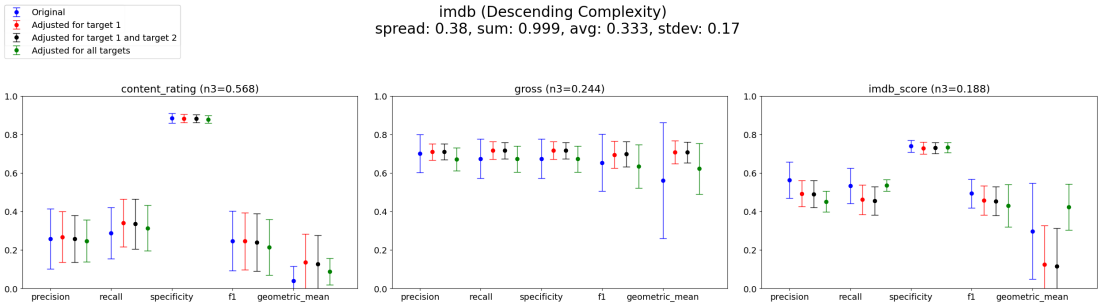

Fig. 40: Performance of the IMDb movies model when applying SMOTE in order of descending complexity.

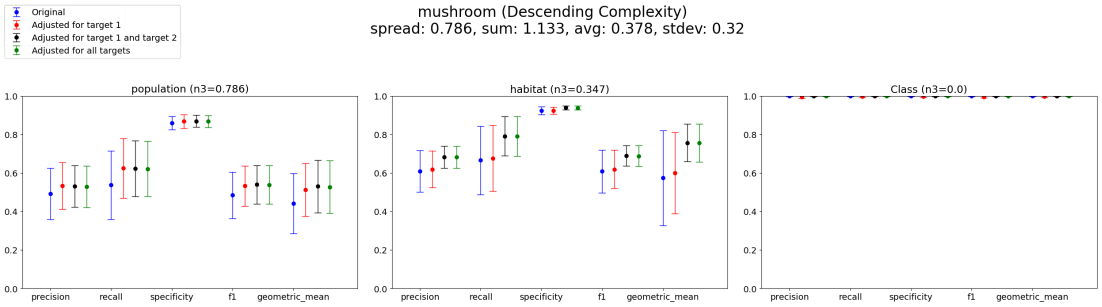

Fig. 41: Performance of the Mushroom model when applying SMOTE in order of descending complexity.

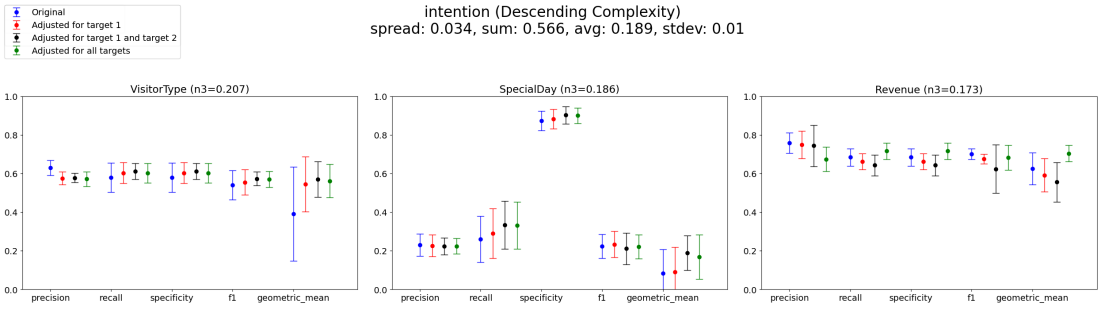

Fig. 42: Performance of the Online Shopping Intention model when applying SMOTE in order of descending complexity.

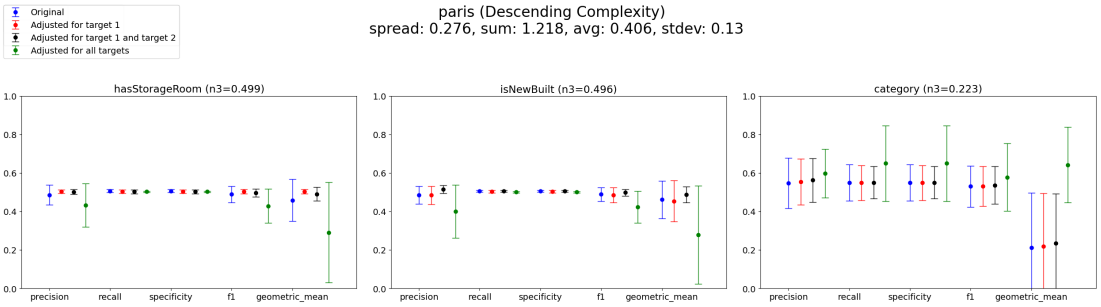

Fig. 43: Performance of the Paris Housing Data model when applying SMOTE in order of descending complexity.

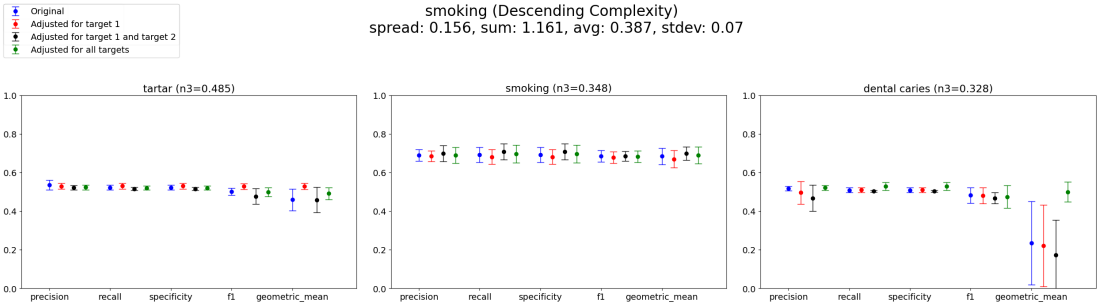

Fig. 44: Performance of the Smoking Health Effects model when applying SMOTE in order of descending complexity.

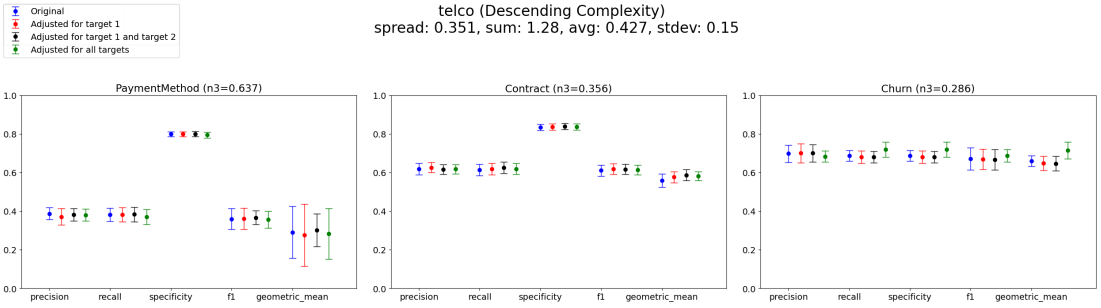

Fig. 45: Performance of the Telco Data model when applying SMOTE in order of descending complexity.

4.5 Appendix V

Performance Difference Between Original and Final Dataset Without Consideration of Target Variable Complexity

Dataset	Target	Precision	Recall	Specificity	F1	Geometric Mean
imdb	1	-8.04	4.7	0.47	-5.19	21.34
imdb	2	-2.75	-0.05	-0.53	-4.39	-1.19
imdb	3	-2.23	0.99	0.99	-0.58	7.96
mushroom	1	-0.01	0.02	0.02	0.0	0.02
mushroom	2	4.11	9.17	1.03	5.97	9.28
mushroom	3	6.62	11.47	1.4	7.36	17.9
census	1	-1.48	-1.77	-1.77	-1.81	-2.25
census	2	-1.87	-0.13	-0.62	-2.43	-3.67
census	3	-1.73	0.6	-0.13	-6.01	1.5
bank	1	-2.67	2.81	2.81	0.35	4.27
bank	2	2.77	0.19	0.19	2.48	20.76
bank	3	-0.58	-0.24	-0.24	1.2	3.57
intention	1	-4.67	2.38	2.38	-0.4	5.55
intention	2	-5.04	1.7	1.7	3.9	14.23
intention	3	-2.31	6.91	2.69	-1.97	6.8
anuran	1	4.34	8.76	0.5	6.07	25.57
anuran	2	-1.18	5.88	-0.0	2.07	20.12
anuran	3	-0.17	5.16	0.1	3.23	7.6
telco	1	-0.98	1.86	1.86	1.2	3.44
telco	2	-0.51	1.26	0.49	0.48	3.82
telco	3	-0.49	0.52	0.09	1.41	3.89
paris	1	2.09	8.86	8.86	5.96	20.84
paris	2	1.68	-0.37	-0.37	-3.65	-7.77
paris	3	1.33	-0.67	-0.67	0.93	3.85
smoking	1	0.01	0.47	0.47	-0.22	0.57
smoking	2	-1.07	0.13	0.13	1.79	5.72
smoking	3	0.33	1.93	1.93	-0.97	26.35
flight	1	-1.1	-2.75	-2.75	-2.7	-3.73
flight	2	-3.6	0.5	0.5	-0.99	2.35
flight	3	0.58	-0.26	-2.63	-1.06	35.79
Average		-0.58	2.19	0.59	0.38	7.95
Average (Target 1)	1	-1.25	2.53	1.28	0.33	7.56
Average (Target 2)	2	-0.75	1.83	0.25	0.52	6.37
Average (Target 3)	3	0.13	2.64	0.35	0.35	11.52

Table 2: test

Performance Difference Between Original and Final Dataset When Sorted in Descending Order of Target Variable Complexity

Performance Difference Between Original and Final Dataset When Sorted in Ascending Order of Target Variable Complexity

4.6 Size of each dataset after smote application

Dataset	Target	Precision	Recall	Specificity	F1	Geometric Mean
imdb	1	-1.07	2.62	-0.64	-3.28	4.75
imdb	2	-3.07	-0.17	-0.17	-1.85	6.06
imdb	3	-11.22	0.21	-0.67	-6.41	12.62
mushroom	1	3.62	8.41	0.87	5.44	8.51
mushroom	2	7.24	12.53	1.54	7.96	18.22
mushroom	3	0.0	0.02	0.02	0.01	0.02
census	1	-0.95	0.91	-0.92	-2.78	4.82
census	2	-3.16	0.39	-0.1	-4.16	1.9
census	3	-5.66	-1.29	-1.29	-2.56	0.89
bank	1	-0.81	-0.43	-0.43	-0.89	-0.02
bank	2	3.64	0.86	0.86	3.94	17.67
bank	3	-4.25	1.42	1.42	-1.55	2.2
intention	1	-5.87	2.37	2.37	3.02	17.01
intention	2	-0.71	6.97	2.65	-0.25	8.37
intention	3	-8.44	3.27	3.27	-1.78	7.89
anuran	1	-1.22	4.26	0.01	1.7	6.44
anuran	2	-2.58	4.9	-0.25	0.06	18.82
anuran	3	0.38	7.95	-0.19	1.6	24.52
telco	1	-0.79	-1.15	-0.41	-0.35	-0.68
telco	2	-0.04	0.56	0.28	0.37	2.34
telco	3	-1.48	3.16	3.16	1.53	5.57
paris	1	-5.39	-0.35	-0.35	-6.05	-16.72
paris	2	-8.48	-0.47	-0.47	-6.64	-18.22
paris	3	5.09	10.01	10.01	4.72	43.1
smoking	1	-1.15	-0.19	-0.19	-0.21	3.24
smoking	2	-0.02	0.44	0.44	-0.29	0.51
smoking	3	0.43	1.97	1.97	-0.7	26.47
flight	1	-1.01	-2.69	-3.36	-2.26	26.57
flight	2	-1.47	-1.57	-1.57	-1.59	-1.65
flight	3	-5.12	2.56	2.56	-1.66	5.1
Average	-	-1.67	2.11	0.64	-0.47	7.38
Average Target 1	1	-1.46	1.37	-0.31	-0.57	5.39
Average Target 2	2	-0.87	2.44	0.32	-0.24	5.4
Average Target 3	3	-3.03	2.93	2.03	-0.68	12.84

Dataset	Target	Precision	Recall	Specificity	F1	Geometric Mean
imdb	1	-8.61	5.69	0.69	-5.38	22.77
imdb	2	-1.44	0.15	0.15	-2.13	7.4
imdb	3	-2.33	0.83	-0.49	-3.56	-0.28
mushroom	1	0.0	0.02	0.02	0.01	0.02
mushroom	2	6.07	11.1	1.26	7.05	17.2
mushroom	3	6.02	9.91	1.38	5.6	7.41
census	1	-0.41	-1.38	-1.38	-1.61	-2.2
census	2	-3.3	0.23	-0.09	-2.89	3.08
census	3	-1.81	-0.96	-0.67	-4.97	-5.3
bank	1	-2.65	2.96	2.96	0.43	4.51
bank	2	2.84	0.28	0.28	2.56	20.7
bank	3	-0.73	-0.4	-0.4	1.06	3.41
intention	1	-5.81	1.38	1.38	-1.03	4.41
intention	2	-2.12	6.7	2.53	-1.81	9.01
intention	3	-5.85	2.51	2.51	3.6	17.56
anuran	1	4.44	8.77	0.5	6.15	25.59
anuran	2	-1.28	5.63	-0.0	1.89	19.85
anuran	3	0.03	5.23	0.1	3.38	7.69
telco	1	-0.96	1.88	1.88	1.22	3.45
telco	2	-0.56	1.19	0.47	0.42	3.77
telco	3	-0.58	0.44	0.07	1.32	3.79
paris	1	2.25	8.89	8.89	6.07	20.87
paris	2	1.65	-0.4	-0.4	-3.68	-7.8
paris	3	1.41	-0.59	-0.59	1.0	3.92
smoking	1	0.97	2.57	2.57	2.09	26.98
smoking	2	0.34	0.97	0.97	-0.34	1.0
smoking	3	-1.05	0.03	0.03	1.46	4.95
flight	1	-2.84	-2.1	-2.1	-2.05	-1.48
flight	2	-1.68	-2.01	-2.01	-2.01	-2.23
flight	3	0.44	-0.43	-2.54	-0.83	35.28
Average	-	-0.58	2.3	0.6	0.43	8.51
Average Target 1	1	-1.36	2.87	1.54	0.59	10.49
Average Target 2	2	0.05	2.38	0.32	-0.09	7.2
Average Target 3	3	-0.45	1.66	-0.06	0.81	7.84

dataset	Original size	First Oversample	second Oversample	Third Oversample	Size increase after one oversamples	Size Increase after Three Oversamples
anuran	5756	14240	28480	35600	2.5	6.2
bank	2472	4374	7902	9592	1.8	3.9
census	36156	54480	222369	1032031	1.5	28.5
flight	103590	117368	195506	321906	1.1	3.1
imdb	2990	7791	21952	24008	2.6	8.0
intention	9700	16426	25210	142032	1.7	14.6
mushroom	4516	5606	14124	34920	1.2	7.7
paris	8000	13996	19974	26150	1.7	3.3
smoking	35642	45256	47238	77814	1.3	2.2
telco	5608	8344	16974	20148	1.5	3.6
Average	19493.6	26171	54520.8	156745.5	1.5	8.0